TP 6 : PROGRAMMATION ORIENTEE OBJET : HERITAGE ET POLYMORPHISME

Langage C++

Consigne : Le TP sera déposé sur Moodle au plus tard une semaine après la séance.

Dans ce TP vous construisez un mini réseau-social.

1. Interfaces et classe abstraites

1.1. Interface « IAimable »

Créez une interface `IAimable` qui définirait un comportement pour les contenus pouvant être aimés. Déclarez les méthodes virtuelles pures suivantes :

- `aimer` : prend un identifiant d'utilisateur (`string`) et enregistre qu'il aime le contenu
- `retirerAime` : annule l'action d'aimer
- `getNombreAimes` : renvoie le nombre total d'utilisateurs qui aiment
- `estAimePar` : vérifie si un utilisateur spécifique aime déjà
- N'oubliez pas le destructeur virtuel par défaut

1.2. Interface « IPartageable »

Créez une interface `IPartageable` pour les contenus pouvant être partagés.

- Déclarez les méthodes virtuelles pures suivantes :
  - `partager` : prend un identifiant d'utilisateur et enregistre le partage
  - `getNombrePartages` : renvoie le nombre total de partages
- N'oubliez pas le destructeur virtuel par défaut

1.3. Classe abstraite « Contenu »

Créez une classe abstraite `Contenu` qui servira de base pour tous les types de contenus.

- Attributs protégés :
  - `id` : identifiant unique du contenu
  - `idAuteur` : identifiant de l'utilisateur qui a créé le contenu
  - `texte` : contenu textuel
  - `motsCles` : liste de mots-clés (vector de `string`)
- Méthodes publiques concrètes :
  - Un constructeur qui initialise les attributs
  - Des getters pour `id`, `idAuteur` et `texte`
  - Un setter pour `texte`
  - `ajouterMotCle` : pour ajouter un mot-clé au contenu
- Méthodes virtuelles pures :
  - `afficher` : pour afficher les détails du contenu

  - `getType` : pour récupérer le type de contenu (`string`)
- N'oubliez pas un destructeur virtuel

2. Classes concrètes

2.1. Classe « Publication »

Implémentez une classe `Publication` qui hérite de `Contenu` et implémente `IAimable`.

- Attributs privés :
  - `utilisateursQuiAiment` : vector stockant les ids des utilisateurs qui aiment
  - `commentaires` : vector de chaînes contenant les commentaires
- Méthodes :
  - Un constructeur qui initialise les attributs en appelant le constructeur de la classe parente
  - L'implémentation des méthodes de l'interface `IAimable`
  - L'implémentation des méthodes abstraites de `Contenu`
  - Une méthode `ajouterCommentaire` : pour ajouter un commentaire
  - Un getter pour récupérer la liste des commentaires

2.2. Classe « Media »

Implémentez une classe `Media` qui hérite de `Contenu` et implémente à la fois `IAimable` et `IPartageable` (héritage multiple).

- Attributs privés :
  - `url` : lien vers le média
  - `typeMedia` : type de média (image, vidéo, audio)
  - `utilisateursQuiAiment` : vector stockant les ids des utilisateurs qui aiment
  - `utilisateursQuiPartagent` : vector stockant les ids des utilisateurs qui partagent
- Méthodes :
  - Un constructeur qui initialise tous les attributs
  - L'implémentation des méthodes des deux interfaces `IAimable` et `IPartageable`
  - L'implémentation des méthodes abstraites de `Contenu`
  - Des getters pour `url` et `typeMedia`

3. Classe « Utilisateur » et système

3.1. Classe « Utilisateur »

Implémentez une classe `Utilisateur` qui représente un utilisateur du réseau social.

- Attributs privés :
  - `id` : identifiant unique

  - `nom` : nom de l'utilisateur
  - `abonnements` : vector d'identifiants des utilisateurs suivis
  - `contenusCrees` : vector de pointeurs vers les contenus créés par l'utilisateur
- Méthodes publiques :
  - Un constructeur qui initialise `id` et `nom`
  - Un destructeur qui libère la mémoire des contenus créés
  - Des getters pour `id` et `nom`
  - `abonner` : pour s'abonner à un autre utilisateur
  - `desabonner` : pour se désabonner
  - `estAbonneA` : pour vérifier si l'utilisateur est abonné à un autre
  - `creerPublication` : pour créer une nouvelle publication
  - `creerMedia` : pour créer un nouveau média
  - `getContenus` : pour récupérer la liste des contenus créés
  - `afficherProfil` : pour afficher les informations du profil

3.2. Classe « ReseauSocial »

Implémentez une classe `ReseauSocial` qui gère l'ensemble du système.

- Attributs privés :
  - `utilisateurs` : map associant les identifiants aux pointeurs d'utilisateurs
  - `contenus` : map associant les identifiants aux pointeurs de contenus
  - `prochainIdContenu` : compteur pour générer des identifiants uniques
- Méthodes publiques :
  - Un constructeur qui initialise les attributs
  - Un destructeur qui libère la mémoire allouée
  - `inscrireUtilisateur` : pour créer et ajouter un nouvel utilisateur
  - `getUtilisateur` : pour récupérer un utilisateur par son id
  - `ajouterContenu` : pour ajouter un contenu au réseau
  - `getContenu` : pour récupérer un contenu par son id
  - `interagir` : pour permettre à un utilisateur d'interagir avec un contenu qui prend en argument l'id de l'utilisateur, l'id du contenu et le type d'interaction (string, aimer ou partager)
  - `afficherFilActualite` : pour afficher le fil d'actualité d'un utilisateur (affiche les publications et médias des utilisateurs auquel il est abonné)
  - `afficherContenuParMotCle` : pour afficher les contenus associés à un mot-clé

4. Programme test

Testez votre réseau social avec le programme suivant :

```
int main() {
```

```
    ReseauSocial reseau;

    // Création d'utilisateurs
    reseau.inscrireUtilisateur("Alice");  // id sera "utilisateur1"
    reseau.inscrireUtilisateur("Bob");    // id sera "utilisateur2"

    // Alice crée du contenu
    Utilisateur* alice = reseau.getUtilisateur("utilisateur1");
    alice->creerPublication("Bonjour tout le monde !");
    alice->creerMedia("Regardez cette photo", "http://exemple.com/image.jpg", "image");

    // Bob s'abonne à Alice et interagit avec son contenu
    Utilisateur* bob = reseau.getUtilisateur("utilisateur2");
    bob->abonner("utilisateur1");

    // Bob aime et partage le contenu d'Alice
    reseau.interagir("utilisateur2", "contenu1", "aimer");  // La publication
    reseau.interagir("utilisateur2", "contenu2", "aimer");  // Le media
    reseau.interagir("utilisateur2", "contenu2", "partager"); // Partage seulement le media

    // Affichage des profils
    std::cout << "Profil d'Alice:" << std::endl;
    alice->afficherProfil();

    std::cout << "\nProfil de Bob:" << std::endl;
    bob->afficherProfil();

    // Affichage du fil d'actualité de Bob
    std::cout << "\nFil d'actualité de Bob:" << std::endl;
    reseau.afficherFilActualite("utilisateur2");

    return 0;
}
```